# CS 285 Assignment 4: LLM RL Report

Xiaolei Chu
Student ID: 3038525739

Spring 2026

## Experimental Context

All figures and numbers below come from the actual exported W&B runs included with the submission. The four required runs were:

- `format_copy + GRPO`
- `format_copy + GR-REINFORCE`
- `math_hard + GR-REINFORCE` using the final passing rerun (`math_hard_reinforce_rerun3`)
- `math_hard + GRPO`

The five extra ablation runs were all additional `format_copy + GRPO` runs with one hyperparameter changed at a time: `ppo_epochs=1`, `kl_coef=0.01`, `kl_coef=0.20`, `clip_eps=0.10`, and `minibatch_size=24, grad_accum_steps=1`.

## 1 Question 1: Approximate KL

Let

$$\Delta(a) = \log \pi_{\text{ref}}(a \mid s) - \log \pi_\theta(a \mid s)$$

for a sampled token $a \sim \pi_\theta(\cdot \mid s)$ at decoding state $s$. The homework uses

$$\hat{k}(a) = e^{\Delta(a)} - \Delta(a) - 1$$

as the sampled-token KL penalty.

The key identity is

$$e^{\Delta(a)} = \frac{\pi_{\text{ref}}(a \mid s)}{\pi_\theta(a \mid s)}.$$

Taking expectation under the current policy gives

$$\mathbb{E}_{a\sim\pi_\theta}\left[e^{\Delta(a)}\right] = \sum_a \pi_\theta(a \mid s)\frac{\pi_{\text{ref}}(a \mid s)}{\pi_\theta(a \mid s)} = \sum_a \pi_{\text{ref}}(a \mid s) = 1.$$

Also,

$$\mathbb{E}_{a\sim\pi_\theta}[\Delta(a)] = \mathbb{E}_{a\sim\pi_\theta}[\log \pi_{\text{ref}}(a \mid s) - \log \pi_\theta(a \mid s)] = -\mathrm{KL}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\text{ref}}(\cdot \mid s)).$$

Therefore,

$$\mathbb{E}[\hat{k}(a)] = \mathbb{E}[e^{\Delta(a)}] - \mathbb{E}[\Delta(a)] - 1 = 1 - (-\mathrm{KL}) - 1 = \mathrm{KL}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\text{ref}}(\cdot \mid s)).$$

So $e^\Delta - \Delta - 1$ is an unbiased sampled-token estimator of the exact KL divergence.

This estimator is attractive because it only needs the log-probability of the *sampled token*, which we already compute during rollout and update. By contrast, the exact token-level KL requires a full sum over the vocabulary:

$$\mathrm{KL}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid s)) = \sum_{a \in \mathcal{V}} \pi_\theta(a \mid s) \log \frac{\pi_\theta(a \mid s)}{\pi_{\mathrm{ref}}(a \mid s)}.$$

For this homework, a typical `math_hard` rollout can have 64 completions per training step and up to 512 generated tokens. With a vocabulary on the order of $1.5 \times 10^5$, the dense tensor needed for exact KL would be about

$$64 \times 512 \times 150{,}000 \approx 4.9 \times 10^9$$

logit values. Even in half precision that is roughly 10 GB just to hold one large distribution tensor, before counting activations, model weights, and optimizer state. The sampled-token estimator avoids that memory and compute cost entirely.

## 2 Question 2: Implementation Order and One Bug

**Implementation order.** I followed the suggested dependency order from the homework:

1. `compute_per_token_logprobs`, `build_completion_mask`, and `approx_kl_from_logprobs`
2. `iter_minibatches`
3. `compute_group_advantages` and `maybe_normalize_advantages`
4. `Reinforce.update`
5. `GRPO.update`

This was the right order in practice: once token-level log-probs and masks were correct, the rollout buffer and the two update rules became much easier to reason about.

**One bug/confusion point.** The most important bug I hit was in `compute_group_advantages`. My first version treated a near-zero reward standard deviation as a batch-level failure case:

$$\text{if any group has } \sigma_g < \varepsilon, \text{ return all-zero advantages.}$$

That was wrong. It meant one "flat" prompt group could zero out the learning signal for the *entire* rollout batch. On `format_copy`, this happened late in training when some prompt groups already had identical rewards, and it caused intermittent dead updates. The correct fix was to apply the fallback *per group*: normalize the stable groups normally, and set only the unstable groups to zero with a `torch.where`-style mask.

The other conceptual confusion point was the completion mask indexing. The log-prob tensor has shape $[B, L-1]$ because position $t$ scores token $x_{t+1}$, so the first completion token lives at mask index `prompt_input_len` $-1$, not at `prompt_input_len`. Once I wrote that down explicitly, the rest of the alignment issues became much easier.

## 3 Question 3: GR-REINFORCE vs. GRPO on `math_hard`

Figure 1 compares the two `math_hard` runs. Over the first 200 training iterations, both runs start from the same pretrained baseline of 22.66% boxed-answer exact match, but GRPO pulls ahead steadily.

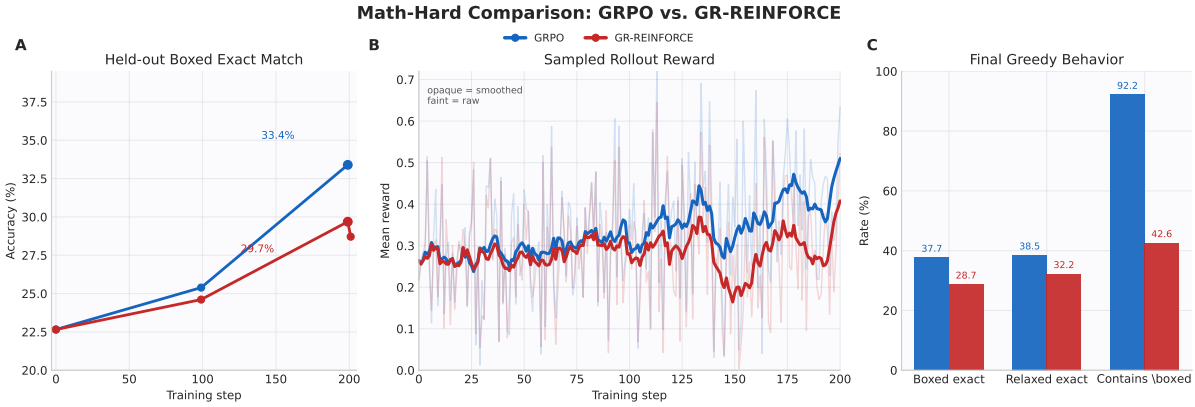


Figure 1: Comparison of the two `math_hard` runs. Panel A shows boxed-answer exact match over the first 200 iterations; Panel B shows sampled rollout reward; Panel C shows the final greedy behavior that matters most for evaluation.

Table 1: Key `math_hard` metrics from the actual W&B runs.

| Run | Step 0 | Step 99 | Step 199 | Final boxed exact | Final boxed-pattern rate | Relaxed-only gap |
|---|---|---|---|---|---|---|
| GR-REINFORCE | 22.66% | 24.61% | 29.69% | 28.71% @ step 201 | 42.58% | 3.52% |
| GRPO | 22.66% | 25.39% | 33.40% | 37.70% @ step 501 | 92.19% | 0.78% |

There are two takeaways.

First, GRPO is better on the actual homework metric. By step 199, GRPO reaches 33.40% boxed exact while GR-REINFORCE reaches 29.69%. GRPO then continues climbing to 37.70% by the end of its longer run, whereas the final passing GR-REINFORCE rerun finishes at 28.71% on step 201.

Second, the difference is *not* just "higher sampled reward." In fact, the GR-REINFORCE run ends with a higher *sampled* rollout reward (0.522 at step 200) than the GRPO run's final-batch reward in the summary, yet its greedy boxed-answer accuracy is worse. The explanation is visible in Panel C: GR-REINFORCE relies much more on the relaxed last-number parser and emits the required `\boxed{}` format much less consistently. Its final boxed-pattern rate is only 42.58%, compared with 92.19% for GRPO. The relaxed-minus-boxed gap is also much larger for GR-REINFORCE (3.52% vs. 0.78%). In other words, GR-REINFORCE is often "numerically close" on sampled rollouts without consistently producing the exact greedy format that the evaluator parses.

This comparison is especially interesting because the provided commands are nearly matched:

- same task and model
- same `batch_size=8`, `group_size=8`, `minibatch_size=8`, `grad_accum_steps=8`
- same learning rate and KL coefficient

The main algorithmic difference is that GRPO reuses the same rollout data with `ppo_epochs=2` and a clipped PPO-style objective, while GR-REINFORCE makes one on-policy pass. The W&B curves show that this extra reuse is not just a theoretical benefit: on `math_hard`, it converts into noticeably better boxed-answer behavior under greedy evaluation.

# 4 Question 4: GRPO Ablations on `format_copy`

All six `format_copy + GRPO` runs eventually reached 100% eval exact match, so the informative differences are in *how fast* they learn and what the diagnostics say about the update dynamics.

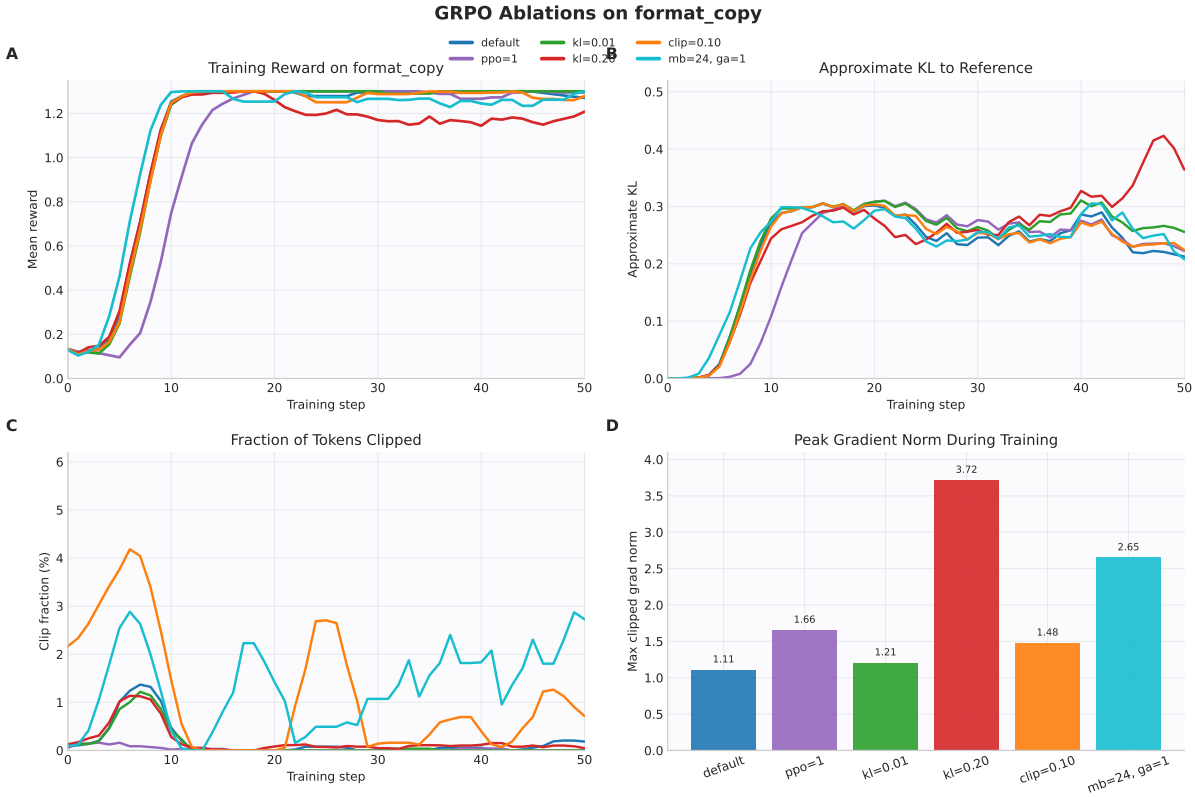


Figure 2: The five GRPO ablations plus the default `format_copy` GRPO run. The easiest task metric saturates for all runs, so the most useful signals are reward speed, KL growth, clipping behavior, and gradient spikes.

Table 2: Summary of the extra `format_copy + GRPO` ablation runs.

| Run | Changed hyperparameter | First step with reward $\geq 1.28$ | Final reward | Final KL | Max clip (%) | Max grad norm |
|---|---|---|---|---|---|---|
| Default | none | 11 | 1.271 | 0.185 | 2.79 | 1.11 |
| `ppo=1` | `ppo_epochs`: $2 \rightarrow 1$ | 16 | 1.300 | 0.203 | 0.45 | 1.66 |
| `kl=0.01` | `kl_coef`: $0.05 \rightarrow 0.01$ | 10 | 1.300 | 0.237 | 2.24 | 1.21 |
| `kl=0.20` | `kl_coef`: $0.05 \rightarrow 0.20$ | 10 | 1.219 | 0.290 | 2.42 | 3.72 |
| `clip=0.10` | `clip_eps`: $0.20 \rightarrow 0.10$ | 11 | 1.273 | 0.211 | 5.05 | 1.48 |
| `mb24, ga1` | `minibatch_size`: $8 \rightarrow 24$; `grad_accum_steps`: $6 \rightarrow 1$ | 8 | 1.300 | 0.178 | 5.60 | 2.65 |

**What mattered most?** The KL coefficient mattered the most. Making it too large (`kl_coef=0.20`) clearly hurt learning: it still touched a high reward early, but it finished substantially lower (1.219 vs. 1.271 for default) and had by far the largest gradient spikes (3.72). On this easy task, the overly strong regularizer fights the policy improvement signal and prevents stable convergence.

**What made learning slower?** Reducing to `ppo_epochs=1` was the clearest slowdown. It still finished perfectly, but it needed 16 steps to reach high reward instead of 11. That matches the

intuition that GRPO's main advantage is sample reuse: with only one pass over each rollout, the method is still PPO-style GRPO, but it extracts less optimization progress from each collected batch.

**What changed the update geometry without hurting final performance?**

- `clip_eps=0.10` roughly doubled the maximum clip fraction (to 5.05%) but still converged almost identically to default.
- `mb24, ga1` reached high reward fastest (step 8) but also produced the highest sustained clipping and much larger gradient norms, so it looks more aggressive rather than clearly better.
- `kl_coef=0.01` converged quickly and cleanly on this task, but it allowed larger final KL drift than the default, so I would trust it less on harder tasks than on `format_copy`.

Overall, the default GRPO configuration from the homework looks sensible: it is not the fastest in every micro-metric, but it avoids the instability of strong KL, avoids the overly aggressive updates of `mb24, ga1`, and learns faster than `ppo=1`.

# 5 Question 5: Qualitative Examples from WandB

The clearest qualitative contrast came from looking at the `math_hard` sample tables around step 109. I intentionally chose two completions for the *same problem*, because that isolates the behavioral difference between the algorithms.

**GRPO example (step 109, reward 1.1).**
**Question:** "Find the value of $a$ that satisfies $293_a + 468_a = 73B_a$, where $B_a = 11_{10}$."
**Observed behavior:** the model converted each base-$a$ number to base 10, solved

$$2a^2 + 9a + 3 + 4a^2 + 6a + 8 = 7a^2 + 3a + 11,$$

reduced it to $a(a - 12) = 0$, discarded the invalid base $a = 0$, and then terminated cleanly with

`\boxed{12}`

**Why this is useful:** it is exactly the behavior the evaluator wants: correct number, correct format, and a short, decisive ending.

**GR-REINFORCE example (step 109, reward 0.1).**
**Question:** the same base-conversion problem above.
**Observed behavior:** this completion did the algebra correctly and reached the sentence "the only valid solution is $a = 12$," but then kept going into extra verification text and never finished with a boxed answer before the sample ended. The relaxed parser still recovered the final number 12, so the sample got the small relaxed bonus, but the boxed parser marked it wrong.
**Why this is useful:** it matches the quantitative gap in Figure 1: GR-REINFORCE often learns the *number* without learning the *greedy boxed format* reliably enough.

These examples helped me trust the metric story. The difference between the two math runs is not that GR-REINFORCE is incapable of doing the algebra; rather, GRPO is much better at turning "I know the answer" into "I output the answer in exactly the parseable format the evaluator rewards."

## Appendix: Sanity-Check Results for the Required Runs

For completeness, both required `format_copy` runs solved the debugging task cleanly: by step 49 they reached 100% exact match, 100% XML-tag rate, and 100% strict XML-only rate. That sanity check made it much easier to interpret the later `math_hard` behavior as a real algorithmic difference rather than a broken training pipeline.